

Clamping Algorithm: Phase 4 Specification

Purpose

Phase 4 of the quarter loop converts a firm's REQUESTED decisions into FEASIBLE actuals. The orchestrator enforces hard resource constraints: a firm cannot spend money it does not have. This document specifies the exact algorithm with pseudocode, edge cases, and test fixtures.

The clamping function is a pure function: given inputs, it produces the same outputs every time. No LLM. No randomness. No side effects.

Inputs and Outputs

```
def clamp_decisions(
    firm: FirmState,
    decisions: RawDecisions,
    expected_revenue: float,
    expected_ar_collection: float,
    available_credit: float,
    params: SimParams,
) -> ClampedResult:
    """
    Convert requested decisions to feasible decisions given available resources.

    Returns a ClampedResult containing:
    - actual decisions (what will be posted to accounting)
    - clamping_log: list of which fields were clamped, original vs. actual
    - solvency_status: "solvent" | "needs_credit" | "default"
    - credit_drawn: amount drawn from revolver this quarter
    """
```

What the firm controls (RawDecisions)

```
@dataclass
class RawDecisions:
    price: float
    production: int
    capex: float
    rd_spend: float
    rd_allocation: dict[str, float] # product/process/delivery, must sum to 1.0
    sga_spend: float
    equity_issuance_request: float
    debt_request: float
    dividends: float
    buybacks: float
```

What the orchestrator computes before calling clamp

```
expected_revenue = decisions.production * decisions.price * collection_rate_same_q
```

```

# NOTE: this is OPTIMISTIC -- assumes 100% of production sells
# The clamping uses this as a cash inflow estimate

expected_ar_collection = firm.accounts_receivable * 1.0
# Assume 100% of prior AR collected this quarter

available_credit = firm.revolver_commitment - firm.revolver_balance
# Already-drawn revolver is not available

```

Output

```

@dataclass
class ClampedResult:
    actual_price: float
    actual_production: int
    actual_capex: float
    actual_rd_spend: float
    actual_sga_spend: float
    actual_dividends: float
    actual_buybacks: float
    credit_drawn: float
    clamping_log: list[str]
    solvency_status: str # "solvent" | "needs_credit" | "default"
    rationale: str # human-readable summary

```

The Algorithm

```

def clamp_decisions(firm, decisions, expected_revenue, expected_ar_collection,
                    available_credit, params):

    log = []

    # =====
    # STEP 0: VALIDATE INPUTS
    # =====

    # Sanitize raw decisions (clip to non-negative)
    raw = sanitize(decisions, log)
    # raw.price >= 0, raw.production >= 0, all spending >= 0
    # If price was negative, log "price clipped to 0"

    # Cap production at capacity
    if raw.production > firm.capacity_units:
        log.append(f"production clamped from {raw.production} to {firm.capacity_units} (capacity)")
        raw.production = firm.capacity_units

    # Validate R&D allocation sums to 1.0 (within tolerance)
    alloc_sum = sum(raw.rd_allocation.values())
    if abs(alloc_sum - 1.0) > 0.01:
        # Renormalize
        raw.rd_allocation = {k: v / alloc_sum for k, v in raw.rd_allocation.items()}

```

```

log.append(f"R&D allocation renormalized from {alloc_sum:.2f} to 1.0")

# =====
# STEP 1: COMPUTE STARTING CASH POSITION
# =====

starting_cash = firm.cash + expected_revenue + expected_ar_collection

# Track equity/debt proceeds (added later if approved by financial agents)
# These are NOT available during clamping; clamping uses pre-financing cash.
# Financing decisions happen in Phase 7 (after clamping).

available = starting_cash

# =====
# STEP 2: COGS (Priority 1 -- mandatory to sell anything)
# =====

# Compute effective unit cost based on REQUESTED production utilization.
# We need to know what the unit cost WOULD be at the requested production.
# This is a slight chicken-and-egg: utilization affects cost, cost affects
# what we can afford. We resolve by computing cost at REQUESTED production,
# then if production must be reduced, we recompute cost at the lower level
# (cost goes UP at lower utilization).

proposed_production = raw.production
iteration = 0
while iteration < 5: # cap iterations; converges quickly
    unit_cost = compute_effective_unit_cost(
        firm=firm,
        production_level=proposed_production,
        params=params,
    )
    cogs_required = proposed_production * unit_cost

    if cogs_required <= available:
        break # affordable

    # Reduce production to what's affordable at this unit cost
    new_production = int(available / unit_cost)
    if new_production == proposed_production:
        break # no further reduction possible
    proposed_production = new_production
    iteration += 1

if proposed_production < raw.production:
    log.append(f"production clamped from {raw.production} to {proposed_production} (insufficient c

actual_production = proposed_production
actual_unit_cost = unit_cost
cogs = actual_production * actual_unit_cost

```

```

available -= cogs

# =====
# STEP 3: MANDATORY OBLIGATIONS (Priority 2)
# =====
# Phase III trial cost, interest on existing debt, taxes due
# If these cannot be met -> default

phase3_cost = params.mandatory_phase3_quarterly_cost # $10M
interest_due = (firm.revolver_balance * firm.revolver_rate
                + firm.long_term_debt * firm.term_debt_rate)
taxes_due = firm.taxes_payable # from prior quarter, paid this quarter

mandatory = phase3_cost + interest_due + taxes_due

if mandatory > available + available_credit:
    # Not enough cash even with full revolver draw
    log.append(f"DEFAULT: mandatory obligations {mandatory} exceed cash + credit")
    return ClampedResult(
        ...,
        solvency_status="default",
        rationale="Insufficient resources for mandatory obligations",
    )

if mandatory > available:
    # Need to draw revolver to cover mandatory
    credit_needed_for_mandatory = mandatory - available
    log.append(f"drawing {credit_needed_for_mandatory} from revolver for mandatory costs")
    available = 0 # all cash used
    available_credit -= credit_needed_for_mandatory
    credit_drawn = credit_needed_for_mandatory
else:
    available -= mandatory
    credit_drawn = 0

# =====
# STEP 4: DISCRETIONARY R&D + SGA + CAPEX (Priority 3 -- pro-rata)
# =====

# NOTE: R&D includes the mandatory $10M Phase III, which we already
# subtracted in Step 3. So discretionary R&D is rd_spend - phase3_cost.

discretionary_rd = max(0, raw.rd_spend - phase3_cost)
if raw.rd_spend < phase3_cost:
    # Firm requested less R&D than the mandatory minimum
    log.append(f"R&D spend raised from {raw.rd_spend} to {phase3_cost} (mandatory minimum)")
    # But Phase III already paid in Step 3; discretionary = 0
    discretionary_rd = 0

discretionary_total = discretionary_rd + raw.sga_spend + raw.capex

```

```

if discretionary_total <= available:
    # Affordable from cash alone
    actual_capex = raw.capex
    actual_discretionary_rd = discretionary_rd
    actual_sga = raw.sga_spend
    available -= discretionary_total
else:
    # Try to use available credit (conservative: not full credit, just enough)
    cash_short = discretionary_total - available

    if cash_short <= available_credit:
        # Can fully fund discretionary by drawing credit
        log.append(f"drawing {cash_short} from revolver for discretionary spending")
        credit_drawn += cash_short
        available_credit -= cash_short
        actual_capex = raw.capex
        actual_discretionary_rd = discretionary_rd
        actual_sga = raw.sga_spend
        available = 0
    else:
        # Pro-rata reduction to fit available cash + credit
        total_resources = available + available_credit
        scale = total_resources / discretionary_total
        log.append(f"pro-rata clamping at {scale:.2%} (insufficient cash + credit)")

        actual_capex = raw.capex * scale
        actual_discretionary_rd = discretionary_rd * scale
        actual_sga = raw.sga_spend * scale

        credit_drawn += available_credit
        available = 0
        available_credit = 0

actual_rd_spend = phase3_cost + actual_discretionary_rd

# =====
# STEP 5: PAYOUTS (Priority 4 -- only from surplus cash, with constraints)
# =====

actual_dividends = 0
actual_buybacks = 0

if raw.dividends > 0 or raw.buybacks > 0:
    # Constraint A: cannot pay dividends if retained earnings are negative
    if firm.retained_earnings <= 0 and raw.dividends > 0:
        log.append(f"dividends blocked: retained earnings {firm.retained_earnings} <= 0")
    elif raw.dividends > 0:
        # Constraint B: must come from surplus cash, not credit
        if available >= raw.dividends:
            actual_dividends = raw.dividends
            available -= raw.dividends

```

```

        else:
            actual_dividends = available
            available = 0
            log.append(f"dividends clamped from {raw.dividends} to {actual_dividends} (limited surplus)")

    # Buybacks: same constraints
    if raw.buybacks > 0:
        if available >= raw.buybacks:
            actual_buybacks = raw.buybacks
            available -= raw.buybacks
        else:
            actual_buybacks = available
            available = 0
            log.append(f"buybacks clamped from {raw.buybacks} to {actual_buybacks} (limited surplus)")

    # =====
    # STEP 6: BUILD RESULT
    # =====

    return ClampedResult(
        actual_price=raw.price, # price is not clamped (just sanitized to >= 0)
        actual_production=actual_production,
        actual_capex=actual_capex,
        actual_rd_spend=actual_rd_spend,
        actual_sga_spend=actual_sga,
        actual_dividends=actual_dividends,
        actual_buybacks=actual_buybacks,
        credit_drawn=credit_drawn,
        clamping_log=log,
        solvency_status="solvent" if credit_drawn == 0 else "needs_credit",
        rationale=summarize_clamping(log),
    )

```

Important Design Notes

Clamping is OPTIMISTIC

Clamping uses expected revenue (production price 0.85) as a cash inflow. If actual sales come in below expected (the environment allocates fewer units), the firm may end the quarter with negative cash. The

settlement step (Phase 8) is what actually checks solvency after market outcomes are known. Clamping is just a sanity filter to prevent obviously infeasible decisions.

This means:

- Clamping passes if decisions COULD be feasible under best-case outcomes
- Settlement enforces solvency under ACTUAL outcomes
- A firm that produces 250 units expecting to sell them all but only sells

100 will have inventory build cost much higher than CFO suggests, and may default in Phase 8

Financing is NOT in Clamping

Equity issuance and debt issuance happen in Phase 7 (after market outcomes). Clamping uses only cash on hand + expected operating cash + available revolver. This means:

- A firm cannot "spend the IPO money" before the IPO actually happens
- New equity raises are realized later in the quarter

For Q1 firms, the IPO sub-sequence (Phase 2) happens BEFORE clamping (Phase 4), so IPO proceeds are already in the cash balance when clamping runs. This is consistent.

Iteration in Step 2

The unit cost depends on production utilization (lower utilization -> higher unit cost). If we have to reduce production for cash, the unit cost goes UP, which means we can afford EVEN LESS production. We iterate until convergence.

In practice this converges in 1-2 iterations because the multiplier is bounded between 1.0 and ~2.0. The cap at 5 iterations is a safety stop.

Phase III is Special

The mandatory \$10M Phase III cost is part of `rd_spend` from the firm's perspective but is treated as a NON-discretionary obligation by clamping. If the firm requested \$25M total R&D, that's \$10M Phase III + \$15M discretionary. If the firm requested \$5M total R&D, the clamping forces it to \$10M (the mandatory minimum) and discretionary R&D is \$0.

What Happens If Firm Can't Pay Phase III

If the firm doesn't even have enough cash + credit for Phase III + interest + taxes, it defaults immediately at Step 3. This is the cleanest default case: the firm is unable to meet its non-negotiable obligations.

Edge Cases (Test Cases)

Edge Case 1: Plenty of Cash, No Clamping

Input: Firm has \$300M cash, requests \$50M total spending, has \$50M revolver available.

Expected: All decisions pass through unchanged. `clamping_log` is empty. `solvency_status` = "solvent". `credit_drawn` = 0.

Edge Case 2: Production Exceeds Capacity

Input: Capacity = 250, requests production = 400.

Expected: production clamped to 250. Log: "production clamped from 400 to 250 (capacity)". Other decisions proceed normally.

Edge Case 3: COGS Exceeds Cash

Input: Cash = \$1M, requests production = 200 at unit cost ~\$14,200 (COGS = \$2.84M). No expected revenue, no AR, no credit.

Expected: production reduced to $\text{int}(\$1\text{M} / \$14,200) = 70$ units. Log shows the clamp. Discretionary spending may be zero.

Edge Case 4: COGS Exceeds Cash After Iteration

Input: Cash = \$500K, requests production = 100 at base cost \$14,200. At $100/250 = 40\%$ utilization, multiplier is 1.6, so effective cost = \$22,720. $\$500\text{K} / \$22,720 = 22$ units. At $22/250 = 8.8\%$ utilization, multiplier jumps to ~2.0, effective cost = \$28,400. $\$500\text{K} / \$28,400 = 17$ units. At 6.8% utilization, multiplier = 2.07, cost = \$29,394. $\$500\text{K} / \$29,394 = 17$ units.

Converges at 17.

Expected: production = 17, unit_cost ~ \$29,394, cogs ~ \$499,698, log shows two iterations.

Edge Case 5: Mandatory Costs Force Default

Input: Cash = \$5M, no credit, mandatory = \$10M (Phase III) + \$2M (interest).

Expected: Step 3 detects \$12M > \$5M + \$0 credit. Returns solvency_status = "default". No further clamping. Log: "DEFAULT".

Edge Case 6: Mandatory Costs Force Revolver Draw

Input: Cash = \$5M, \$50M revolver available, mandatory = \$12M.

Expected: \$7M drawn from revolver. credit_drawn = 7M. Discretionary spending continues with remaining \$43M revolver capacity.

Edge Case 7: Pro-Rata Clamping

Input: After mandatory, \$10M cash + \$20M credit = \$30M available. Firm requests \$20M capex + \$20M discretionary R&D + \$10M SGA = \$50M discretionary.

Expected: scale = 30/50 = 0.6. actual_capex = \$12M, actual_rd_disc = \$12M, actual_sga = \$6M. credit_drawn includes \$20M revolver draw. Log: "pro-rata clamping at 60.00%".

Edge Case 8: Dividend Blocked by Negative RE

Input: Cash = \$50M after all spending (surplus). RE = -\$30M. Dividends requested = \$5M.

Expected: actual_dividends = 0. Log: "dividends blocked: retained earnings -30000000 <= 0".

Edge Case 9: Dividend Limited by Surplus

Input: Surplus after spending = \$3M. RE = \$20M (positive). Dividends requested = \$5M.

Expected: actual_dividends = \$3M. Log: "dividends clamped from 5000000 to 3000000 (limited surplus)".

Edge Case 10: Negative Price

Input: price = -1000.

Expected: price sanitized to 0. Log: "price clipped to 0". Production proceeds normally but revenue will be 0 (handled in accounting).

Edge Case 11: R&D Below Phase III Minimum

Input: rd_spend = \$5M (firm trying to skip Phase III).

Expected: rd_spend forced to \$10M (Phase III minimum). discretionary_rd = 0. Log: "R&D spend raised from 5000000 to 10000000 (mandatory minimum)".

Edge Case 12: R&D Allocation Doesn't Sum to 1.0

Input: rd_allocation = {"product": 0.5, "process": 0.3, "delivery": 0.3} (sums to 1.1).

Expected: Renormalized to {0.4545, 0.2727, 0.2727}. Log: "R&D allocation renormalized from 1.10 to 1.0".

Edge Case 13: Q1 IPO Just Closed -- Massive Cash

Input: Just-IPO'd firm has \$350M cash, no AR, no production yet. Requests modest production = 100, R&D = \$30M, SGA = \$15M, capex = \$25M.

Expected: All affordable. No clamping. Log empty. Solvent.

Edge Case 14: Optimistic Revenue, Settlement Will Fail

Input: Firm has \$5M cash, production = 200, price = \$90K. Expected revenue = $200 \times 90,000 \times 0.85 = \15.3M . Total available for clamping = \$20.3M. Spending requested fits in \$20.3M.

Expected: Clamping passes. But if actual units_sold = 50 (only \$4.25M revenue, vs. expected \$17M), settlement (Phase 8) will detect end-of-quarter cash < 0 and trigger default.

This is the expected behavior: clamping is optimistic; settlement is the actual solvency gate.

Edge Case 15: Zero Production

Input: production = 0.

Expected: COGS = 0. No iteration needed. Mandatory obligations still checked. If firm requests R&D and SGA, they proceed normally if affordable. Inventory stays at prior level (no production, no consumption from production).

Test Fixture Format

```
# tests/fixtures/clamping_cases.py

from dataclasses import dataclass

@dataclass
class ClampingTestCase:
    name: str
    firm: FirmState
    decisions: RawDecisions
    expected_revenue: float
    expected_ar_collection: float
    available_credit: float
    expected_result: ClampedResult

CASES = [
    ClampingTestCase(
        name="case_01_no_clamping_needed",
        firm=FirmState(cash=300_000_000, capacity_units=250, ...),
        decisions=RawDecisions(
            price=92_000, production=200,
            capex=15_000_000, rd_spend=28_000_000,
            rd_allocation={"product": 0.6, "process": 0.25, "delivery": 0.15},
            sga_spend=14_000_000, dividends=0, buybacks=0,
            equity_issuance_request=0, debt_request=0,
        ),
        expected_revenue=15_640_000,
        expected_ar_collection=2_565_000,
        available_credit=0,
        expected_result=ClampedResult(
```

```

        actual_price=92_000,
        actual_production=200,
        actual_capex=15_000_000,
        actual_rd_spend=28_000_000,
        actual_sga_spend=14_000_000,
        actual_dividends=0,
        actual_buybacks=0,
        credit_drawn=0,
        clamping_log=[],
        solvency_status="solvent",
        rationale="No clamping required",
    ),
),

ClampingTestCase(
    name="case_02_production_capped_at_capacity",
    firm=FirmState(cash=300_000_000, capacity_units=250, ...),
    decisions=RawDecisions(production=400, ...),
    expected_result=ClampedResult(
        actual_production=250,
        clamping_log=["production clamped from 400 to 250 (capacity)"],
        ...
    ),
),

ClampingTestCase(
    name="case_05_mandatory_forces_default",
    firm=FirmState(
        cash=5_000_000,
        revolver_balance=0,
        revolver_commitment=0,
        long_term_debt=100_000_000,
        term_debt_rate=0.025, # 2.5%/q -> $2.5M interest
        taxes_payable=0,
        ...
    ),
    decisions=RawDecisions(...),
    expected_revenue=0,
    expected_ar_collection=0,
    available_credit=0,
    expected_result=ClampedResult(
        solvency_status="default",
        rationale="Insufficient resources for mandatory obligations",
    ),
),

# ... 12 more cases

```

```

]

```

```

def test_all_clamping_cases():

```

```

for case in CASES:
    result = clamp_decisions(
        firm=case.firm,
        decisions=case.decisions,
        expected_revenue=case.expected_revenue,
        expected_ar_collection=case.expected_ar_collection,
        available_credit=case.available_credit,
        params=DEFAULT_PARAMS,
    )
    assert_clamped_result_equal(result, case.expected_result, tol=1.0), \
        f"Failed: {case.name}"

```

Helper Function: compute_effective_unit_cost

```

def compute_effective_unit_cost(firm, production_level, params):
    """
    Compute the effective per-unit cost for a given production level.
    Includes process R&D reduction and capacity utilization multiplier.
    """
    # Base cost for this generation, after process R&D
    process_reduction = 0.22 * (1 - exp(-firm.rd_cumulative_process / 120_000_000))
    base_cost = params.gen_base_cogs[firm.product_generation] * (1 - process_reduction)

    # Capacity utilization multiplier
    if firm.capacity_units == 0:
        return float("inf") # cannot produce without capacity

    util = production_level / firm.capacity_units

    if util >= 0.90:
        multiplier = 1.00
    elif util >= 0.70:
        multiplier = 1.00 + 0.5 * (0.90 - util)
    elif util >= 0.50:
        multiplier = 1.10 + 1.0 * (0.70 - util)
    elif util >= 0.30:
        multiplier = 1.30 + 1.5 * (0.50 - util)
    else:
        multiplier = 1.60 + 2.0 * (0.30 - util)

    return base_cost * multiplier

```

Validation After Clamping

After clamping, the orchestrator runs sanity checks:

```

def validate_clamped(result, firm):
    assert result.actual_price >= 0
    assert result.actual_production >= 0
    assert result.actual_production <= firm.capacity_units

```

```
assert result.actual_capex >= 0
assert result.actual_rd_spend >= params.mandatory_phase3_quarterly_cost \
    or result.solvency_status == "default"
assert result.actual_sga_spend >= 0
assert result.actual_dividends >= 0
assert result.actual_buybacks >= 0
assert result.credit_drawn >= 0
assert result.credit_drawn <= firm.revolver_commitment - firm.revolver_balance
assert result.solvency_status in {"solvent", "needs_credit", "default"}
```

What Clamping Does NOT Do

- It does not actually move money. The accounting postings (Phase 6) do that.
- It does not check if production will sell. The environment (Phase 5) does that.
- It does not approve financing. Financial agents (Phase 7) do that.
- It does not finalize defaults. Settlement (Phase 8) does that based on actual cash.

Clamping is just a feasibility filter -- a translator from "what the firm wants to do" to "what the firm can plausibly try to do given resources on hand."